

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении практической работы
по дисциплине «Технологии разработки программного
обеспечения»

Разработка клиент-серверного приложения, вычисляющего
длину минимального пути в неориентированном графе

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Силиненко А. В. _____

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Спецификация требований	5
1.1 Общее описание задачи	5
1.2 Описание интерфейса	5
1.2.1 Входные данные	5
1.2.2 Выходные данные	5
1.3 Функциональные требования	7
1.3.1 Сетевое взаимодействие	7
1.3.2 Ввод данных	8
1.3.3 Проверка корректности и данных	8
1.3.4 Вычисление минимального пути	9
1.3.5 Вывод результатов	9
1.3.6 Завершение работы программы	9
1.4 Требования к тестированию	9
1.4.1 Тестирование работы приложения по протоколу TCP	9
1.4.2 Работа приложения по протоколу UDP	10
1.4.3 Ввод исходных данных с клавиатуры	10
1.4.4 Ввод исходных данных из файла	10
1.4.5 Обработка граничных значений области допустимых значений графа	10
1.4.6 Обработка значений из середины области допустимых значений графа	11
2 Проектирование приложения	12
2.1 Перечень модулей	12
2.1.1 Модули клиентской части	12
2.1.2 Модули серверной части	13
2.2 Алгоритмы работы частей приложения при использовании протокола UDP	14
2.2.1 Алгоритм работы клиентской части	14
2.2.2 Алгоритм работы серверной части	14
2.3 Форматы структур, передаваемые между клиентом и сервером	14
3 Разработка программы	18
3.1 Используемая среда разработки	18
3.2 Используемый компилятор	18
3.3 Основные технологии, используемые в приложении	18
3.3.1 Механизм межпроцессного взаимодействия	18
3.3.2 Механизм мультимплексирования ввода и вывода	19

3.3.3	Примеры запуска и работы программы	19
4	Тестирование	20
4.1	Средства разработки тестирующего приложения	20
4.2	Протоколы тестирования	20
	Заключение	21
	Приложение А. «Исходный код клиентской части приложения»	22
	Приложение Б. «Исходный код серверной части приложения»	31
	Приложение В. «Исходный код заголовочных файлов»	40
	Приложение Г. «Исходный код файлов реализации»	43
	Приложение Д. «Исходный код тестирующего приложения»	55
	Приложение Е. «Протоколы тестирования»	60

Введение

Современные программные системы всё чаще строятся на основе клиент-серверных архитектур, что обусловлено ростом объёмов обрабатываемых данных и необходимостью удалённого взаимодействия между компонентами программного обеспечения. Одной из ключевых технологий при разработке подобных систем является использование сетевых механизмов межпроцессного взаимодействия, в частности сокетов TCP/IP, позволяющих организовать обмен данными между приложениями, работающими на различных вычислительных узлах.

Работа посвящена разработке клиент-серверного приложения, предназначенного для вычисления длины кратчайшего пути в неориентированном взвешенном графе. В рамках работы реализовано сетевое взаимодействие между клиентской и серверной частями приложения с поддержкой протоколов TCP и UDP, а также алгоритм Беллмана–Форда для поиска минимального пути между заданными вершинами графа.

Актуальность данной работы обусловлена необходимостью получения практических навыков разработки приложений в операционных системах семейства Linux, а также освоения принципов построения межпроцессного взаимодействия. Задачи поиска кратчайших путей в графах широко применяются в различных прикладных областях, таких как компьютерные сети, транспортные системы, логистика и анализ данных.

Цель работы: разработать клиент-серверное приложение, вычисляющее длину минимального пути в неориентированном графе.

Задачи работы:

- получение базовых знаний по разработке приложений в ОС Linux;
- получение знаний и практических навыков реализации межпроцессных взаимодействий с помощью механизма сокетов;
- получение знаний по работе с графами;
- разработка клиент-серверного приложения.
- разработка автоматических тестов, обеспечивающих тестирование разработанного клиент-серверного приложения.

1 Спецификация требований

1.1 Общее описание задачи

Назначение: клиент-серверное приложение, вычисляющее длину минимального пути в неориентированном графе.

Язык реализации: Си.

Архитектура: x64.

Операционная система: Linux Ubuntu 24.04.3 LTS.

1.2 Описание интерфейса

1.2.1 Входные данные

1.2.1.1 Данные, вводимые пользователем через клиент

1.2.1.1.1 Цепочка символов в формате $(a, b, w_{ab}), (b, c, w_{bc})$, где a, b, c являются целыми числами от 1 до N , а w является весом ребра – список рёбер графа.

1.2.1.1.2 Целое число от 1 до N – индекс начальной вершины пути.

1.2.1.1.3 Целое число от 1 до N – индекс конечной вершины пути.

1.2.1.2 Входные данные клиента

1.2.1.2.1 Тип подключения $\langle \text{TCP} | \text{UDP} \rangle$.

1.2.1.2.2 IP-адрес.

1.2.1.2.3 Порт серверной части.

1.2.1.2.4 Строка с ответом от сервера, полученная через сокет.

1.2.1.2.5 Данные, вводимые пользователем с клавиатуры.

1.2.1.2.6 Данные, загруженные из файла.

1.2.1.3 Входные данные сервера

1.2.1.3.1 Тип подключения $\langle \text{TCP} | \text{UDP} \rangle$.

1.2.1.3.2 Структура данных, содержащая описание графа и отправленная клиентом через сокет.

1.2.2 Выходные данные

1.2.2.1 Выходные данные клиента, передаваемые серверу через сокет

1.2.2.1.1 Структура данных, описывающая граф.

- 1.2.2.1.2 Индекс начальной вершины пути.
- 1.2.2.1.3 Индекс конечной вершины пути.
- 1.2.2.1.4 Сообщение об успешной отправке данных.
- 1.2.2.2 Выходные данные клиента, отображаемые в пользовательском интерфейсе
 - 1.2.2.2.1 Сообщение об успешной отправке данных.
 - 1.2.2.2.2 Строка формата «Минимальный путь от вершины N до вершины M составляет L».
 - 1.2.2.2.3 Перечень вершин, составляющих кратчайший путь.
 - 1.2.2.2.4 Информационные сообщения и ошибки.
- 1.2.2.3 Выходные данные сервера, передаваемые клиенту через сокет
 - 1.2.2.3.1 Вычисленное значение минимального пути от одной заданной вершины до другой.
 - 1.2.2.3.2 Сообщения об ошибках.
- 1.2.2.4 Выходные данные сервера, отображаемые в консоли сервера
 - 1.2.2.4.1 Сообщение об успешном получении данных от клиента.
 - 1.2.2.4.2 Сообщение об успешной отправке результата клиенту.
 - 1.2.2.4.3 Сообщения о состоянии сервера (запуск, остановка, ошибки соединения).
- 1.2.2.5 Информационные сообщения
 - 1.2.2.5.1 Приветственное сообщение «=== Вычисление пути минимальной длины в неориентированном графе ===».
 - 1.2.2.5.2 Приглашение «Введите количество вершин в графе (одно целое число от 6 до 100)».
 - 1.2.2.5.3 Приглашение «Перечислите рёбра графа в формате (вершина 1, вершина 2, вес ребра 1-2)».
 - 1.2.2.5.4 Приглашение «Введите индекс начальной вершины (целое число от 1 до N)».
 - 1.2.2.5.5 Приглашение «Введите индекс конечной вершины (целое число от 1 до N)».
 - 1.2.2.5.6 Подсказка «Введите \help, чтобы открыть справочник команд».

1.2.2.5.7 Ошибки:

- (a) «Ошибка: некорректное значение. Введите целое число от 6 до 100.»
- (b) «Ошибка: некорректное значение. Введите целое число от 1 до N.»
- (c) «Ошибка: некорректный формат ввода. Введите в формате (1, 2, 3).»
- (d) «Ошибка: не удалось установить соединение с сервером.»
- (e) «Ошибка: не удалось получить данные от сервера.»
- (f) «Ошибка: не удалось отправить данные на сервер.»
- (g) «Ошибка: неверный формат IP-адреса.»
- (h) «Ошибка: номер порта должен быть в диапазоне от 1 до 65535.»

1.2.2.5.8 Сообщение «Потеряна связь с сервером» после трех неудачных попыток клиента отправки данных серверу.

1.2.2.5.9 Сообщение «Потеряна связь с клиентом» при потере связи с клиентом у сервера.

1.2.2.5.10 Сообщение «Данные отправлены» при первой отправке данных клиентом серверу.

1.2.2.5.11 Сообщение «Данные получены» при получении данных от клиента сервером.

1.2.2.5.12 Сообщение «Не удалось отправить данные. Производится повторная отправка» в случае, когда подтверждение от сервера не получено в течение 3 секунд.

1.2.2.6 Хранимые данные

1.2.2.6.1 Структура данных графа: количество вершин N, количество ребер M, матрица весов ребер графа.

1.2.2.6.2 Сетевые данные: дескриптор сокета, адрес сервера, информация о клиентах.

1.3 Функциональные требования

1.3.1 Сетевое взаимодействие

Клиент и сервер должны поддерживать взаимодействие по протоколам TCP и UDP. Выбор протокола должен задаваться параметром при запуске приложения. Порт на стороне сервера произвольный.

При взаимодействии по протоколу UDP необходимо обеспечить надежную доставку полезной нагрузки:

- подтверждение прием сообщения с полезной нагрузкой через отправку служебного сообщения;
- контроль доставки отправленного сообщения с полезной нагрузкой путем ожидания и приема подтверждающего служебного сообщения;
- если подтверждение о получении не приходит в течение 3 с – повторно отправить сообщение с полезной нагрузкой;
- если сообщение с полезной нагрузкой отправлено 3 раза без получения подтверждения, то клиентская часть завершает работу.

Приложение должно функционировать как при запуске клиентской и серверной части на одном компьютере, так и на разных сетевых узлах. В качестве параметра клиентская часть приложения должна принимать IP-адрес и порт серверной части. При запуске приложения на одном компьютере клиентская часть должна использовать в качестве IP-адреса сервера адрес 127.0.0.1.

1.3.2 Ввод данных

Клиент должен запрашивать у пользователя данные о графе и пути и передавать их на сервер. Клиент должен принимать с клавиатуры или загружать из файла размерность графа, структуру графа и индексы вершин.

Сервер должен корректно принимать и интерпретировать передаваемые клиентом данные.

1.3.3 Проверка корректности и данных

Клиент должен проверять, являются ли введенные данные корректными согласно следующим правилам:

- $6 \leq N \leq 100$, где N – количество вершин;
- $6 \leq M \leq \frac{n(n-1)}{2}$, где M – количество рёбер;
- $1 \leq V_{start} \leq N$, $1 \leq V_{end} \leq N$, где V_{start}, V_{end} – индексы стартовой и конечной вершин.
- $W_i > 0$, где W_i – вес i -й вершины графа.

Сервер должен повторно проверять данные на корректность перед выполнением вычислений.

1.3.4 Вычисление минимального пути

Сервер должен реализовывать алгоритм Беллмана-Форда для поиска кратчайшего пути во взвешенном неориентированном графе.

Алгоритм должен корректно обрабатывать ситуации, когда путь между заданными вершинами отсутствует, или когда стартовая и конечная вершины совпадают.

Результатом работы программы является целое число, являющееся длиной найденного кратчайшего пути, и перечень вершин, составляющих кратчайший путь, или сообщение о том, что путь между заданными вершинами отсутствует.

1.3.5 Вывод результатов

Сервер должен направлять результат вычислений клиенту. Клиент должен выводить полученный результат пользователю.

1.3.6 Завершение работы программы

При вводе пользователем команды «exit», клиент должен завершать работу в штатном режиме. Сервер должен обрабатывать сигнал и завершать свою работу.

1.4 Требования к тестированию

Требуется разработать автоматические тесты, обеспечивающие тестирование разработанного клиент-серверного приложения. Тестирование должно выполняться автоматически без участия пользователя и включать проверку сетевого взаимодействия, обработки входных данных и корректности вычисления длины минимального пути в графе.

1.4.1 Тестирование работы приложения по протоколу ТСР

Необходимо проверять, что:

- 1.4.1.1 Клиент успешно устанавливает соединение с сервером.
- 1.4.1.2 Сервер успешно запускается и принимает входящие ТСР-подключения.
- 1.4.1.3 Данные о графе и начальная и конечная вершины пути передаются от клиента серверу.
- 1.4.1.4 Сервер выполняет вычисление длины минимального пути.
- 1.4.1.5 Клиент корректно получает и отображает результат вычислений.

1.4.2 Работа приложения по протоколу UDP

Необходимо проверять:

- 1.4.2.1 Корректность передачи данных без установления постоянного соединения.
- 1.4.2.2 Повторную отправку данных клиентом при отсутствии подтверждения.
- 1.4.2.3 Корректное завершение работы клиента при превышении допустимого числа попыток передачи и корректность получаемого результата вычислений.

1.4.3 Ввод исходных данных с клавиатуры

Необходимо проверять, что:

- 1.4.3.1 Приложение корректно принимает данные о графе в заданном формате.
- 1.4.3.2 Корректно введённые данные успешно передаются серверу и обрабатываются.

1.4.4 Ввод исходных данных из файла

Необходимо обеспечить проверку того, что:

- 1.4.4.1 Приложение корректно открывает и читает файл с данными.
- 1.4.4.2 Данные, считанные из файла, корректно используются для вычисления минимального пути.

1.4.5 Обработка граничных значений области допустимых значений графа

Необходимо проверять, что:

- 1.4.5.1 Приложение корректно работает при минимально допустимом количестве вершин графа ($N = 6$).
- 1.4.5.2 Приложение корректно работает при максимально допустимом количестве вершин графа ($N = 100$).
- 1.4.5.3 Клиент отклоняет запрос, когда количество вершин ниже или выше области допустимых значений ($N = 5$ и $N = 101$).

1.4.6 Обработка значений из середины области допустимых значений графа

Необходимо проверять, что:

- 1.4.6.1 Приложение корректно обрабатывает графы размерности из середины области допустимых значений ($N = 53$).
- 1.4.6.2 Сервер выполняет корректное вычисление длины минимального пути.
- 1.4.6.3 Клиент получает и отображает корректный результат вычислений.

2 Проектирование приложения

2.1 Перечень модулей

Клиент-серверное приложение реализовано в виде набора модулей, каждый из которых отвечает за отдельную функциональную область.

2.1.1 Модули клиентской части

2.1.1.1 Основной модуль клиента

Назначение модуля:

- управление жизненным циклом клиентского приложения;
- обработка аргументов командной строки (IP-адрес, порт, тип протокола);
- инициализация сетевого взаимодействия;
- запуск логики ввода данных, передачи запроса серверу и получения результата.

2.1.1.2 Модуль ввода и обработки данных

Назначение модуля:

- ввод данных о графе с клавиатуры или загрузка из файла;
- разбор строкового представления рёбер графа;
- проверка корректности вводимых данных (количество вершин, рёбер, веса, индексы вершин);
- преобразование пользовательского ввода во внутреннюю структуру данных графа.

2.1.1.3 Сетевой модуль

Назначение модуля:

- установление соединения с сервером по TCP или UDP;
- отправка данных графа и параметров поиска кратчайшего пути;
- получение ответа от сервера;

- реализация логики подтверждения доставки сообщений при работе по протоколу UDP;
- повторная отправка данных при отсутствии подтверждения;
- обработка сетевых ошибок и завершение соединения.

2.1.1.4 Модуль пользовательского интерфейса

Назначение модуля:

- вывод приветственных сообщений;
- отображение приглашений к вводу данных;
- вывод результатов вычислений;
- отображение информационных сообщений и сообщений об ошибках;
- вывод справочной информации.

2.1.2 Модули серверной части

2.1.2.1 Основной модуль сервера

Назначение модуля:

- инициализация серверного приложения;
- создание и привязка сокета к заданному порту;
- обработка сигналов завершения работы;
- запуск основного цикла ожидания и обработки клиентских запросов.

2.1.2.2 Модуль обработки графов

Назначение модуля:

- хранение структуры графа;
- проверка корректности полученных от клиента данных;
- реализация алгоритма Беллмана–Форда для поиска кратчайшего пути;
- формирование результата в виде длины минимального пути и последовательности вершин.

2.1.2.3 Сетевой модуль сервера

Назначение модуля:

- приём входящих TCP и UDP сообщений;
- отправка подтверждений получения данных при работе по протоколу UDP;
- передача результатов вычислений клиенту;
- обработка ошибок сетевого взаимодействия.

2.2 Алгоритмы работы частей приложения при использовании протокола UDP

2.2.1 Алгоритм работы клиентской части

Алгоритм работы клиентской части при использовании протокола UDP приведен на рис. 1

2.2.2 Алгоритм работы серверной части

Алгоритм работы серверной части при использовании протокола UDP приведен на рис. 2.

2.3 Форматы структур, передаваемые между клиентом и сервером

Для обмена данными между клиентской и серверной частями используется сериализованная структура графа.

Передаваемая клиентом серверу структура включает:

- количество вершин графа N – целочисленное значение типа `int`;
- матрицу весов рёбер размером $N \times N$ – вектор троек целочисленных значений типа `int`. Каждая тройка содержит индексы вершин, соединяемые ребром, а также вес этого ребра;
- индекс начальной вершины – целочисленное значение типа `int`;
- индекс конечной вершины – целочисленное значение типа `int`.

В случае отсутствия ребра между вершинами соответствующий элемент матрицы содержит значение, обозначающее бесконечность. Это значение хранится как статическая константная переменная типа `int` и равняется `numeric_limits<int>::max()`.

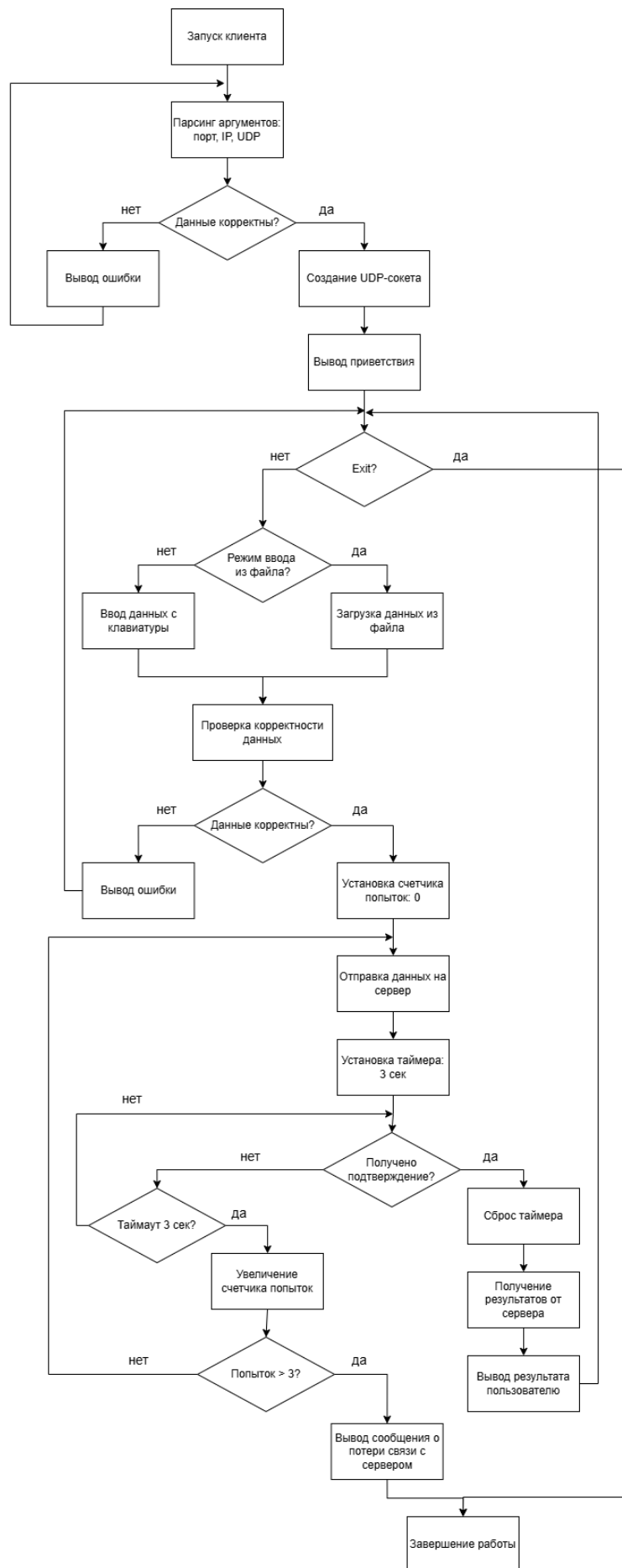


Рис. 1: Алгоритм работы клиентской части UDP

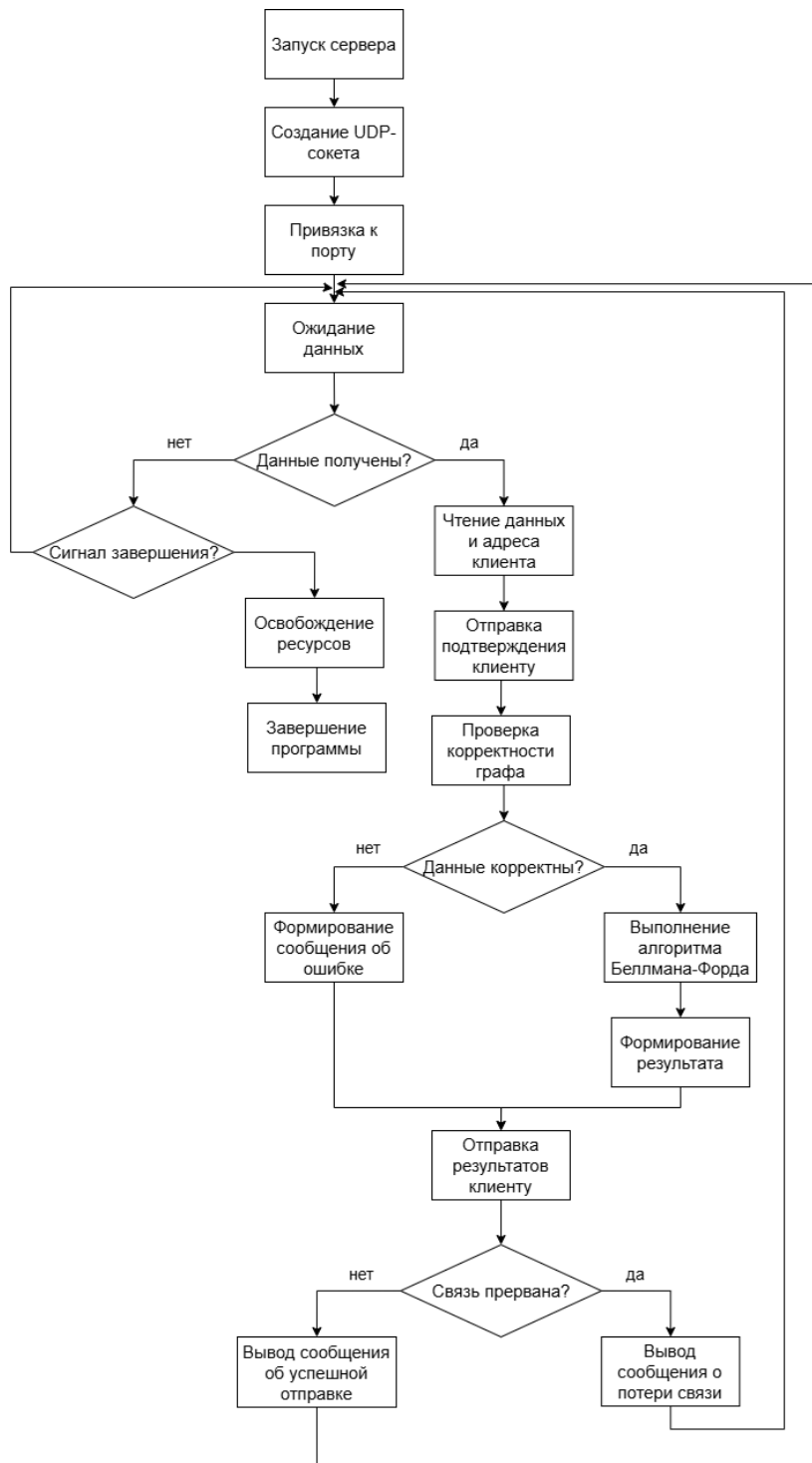


Рис. 2: Алгоритм работы серверной части UDP

Результат, передаваемый сервером клиенту, включает:

- значение типа `bool`, означающее нахождение или отсутствие минимального пути;
- строку с сообщением об ошибке в случае, когда кратчайший путь не найден;
- длину минимального пути – целочисленное значение типа `int`;
- последовательность вершин кратчайшего пути – вектор значений типа `int`, представляющими собой индексы вершин, включенных в путь.

3 Разработка программы

3.1 Используемая среда разработки

Разработка клиент-серверного приложения осуществлялась в операционной системе Linux Ubuntu 24.04.3 LTS.

В качестве среды разработки использовался текстовый редактор Visual Studio Code. Сборка, запуск и отладка приложения выполнялись из стандартного терминала Ubuntu с использованием средств командной строки.

3.2 Используемый компилятор

Для компиляции исходного кода использовался компилятор GNU g++, входящий в состав пакета GNU Compiler Collection (GCC).

Сборка проекта осуществляется с использованием утилиты make и файла сборки Makefile. В процессе сборки применяется стандарт языка C++17.

Сборка клиентской и серверной частей приложения выполняется командой `make`. В результате сборки формируются исполняемые файлы `client` и `server`.

3.3 Основные технологии, используемые в приложении

3.3.1 Механизм межпроцессного взаимодействия

Для организации взаимодействия между клиентской и серверной частями приложения используется механизм сетевых сокетов TCP/IP. Приложение поддерживает работу как по протоколу TCP, так и по протоколу UDP, выбор протокола осуществляется параметром командной строки при запуске клиента.

В ходе работы используются следующие основные системные вызовы:

- `socket()` – создание сокета;
- `bind()` – привязка сокета к IP-адресу и порту на стороне сервера;
- `connect()` – установка соединения с сервером TCP;
- `listen()` – перевод TCP-сокета в режим ожидания входящих подключений;
- `accept()` – принятие TCP-подключения от клиента;
- `send()`, `recv()` – передача и приём данных по протоколу TCP;
- `sendto()`, `recvfrom()` – передача и приём данных по протоколу UDP;

- `close()` – закрытие сокета.

При работе по протоколу UDP в приложении реализован механизм надёжной доставки сообщений, включающий подтверждение получения данных сервером и повторную отправку сообщений клиентом при отсутствии подтверждения в течение 3 секунд.

3.3.2 Механизм мультиплексирования ввода и вывода

Для обеспечения возможности одновременной работы сервера с несколькими клиентами в серверной части используется механизм мультиплексирования ввода/вывода.

Мультиплексирование реализовано с помощью системного вызова `select`. Этот механизм позволяет серверу отслеживать активность нескольких сокетов и принимать и обрабатывать запросы от разных клиентов.

3.3.3 Примеры запуска и работы программы

После успешной сборки приложения запуск серверной части осуществляется командой `./server <PORT>`. Если не ввести номер порта, то сервер будет запущен с портом 8080.

Запуск клиентской части приложения выполняется командой `./client <IP> <PORT> <PROTOCOL>`, где `<IP>` — IP-адрес сервера, `<PORT>` — номер порта сервера, `<PROTOCOL>` — используемый протокол (TCP или UDP).

После запуска сервер переходит в режим ожидания входящих подключений. Клиент выводит приветственное сообщение, принимает от пользователя данные о графе (с клавиатуры или из файла), передаёт их серверу и отображает полученный результат вычисления длины минимального пути и последовательности вершин.

Исходный код программы приведен в приложениях [А](#), [Б](#), [В](#), [Г](#).

4 Тестирование

Тестирование клиент-серверного приложения проводилось с использованием отдельного тестирующего приложения, предназначенного для автоматизации проверки корректности работы клиентской и серверной частей, а также сетевого взаимодействия между ними.

4.1 Средства разработки тестирующего приложения

Тестирующее приложение разработано с использованием языка сценариев Exрrest, предназначенного для автоматизации интерактивных консольных программ.

Exрrest является расширением языка Tcl и широко применяется в операционных системах семейства Linux для автоматического ввода данных в консольные приложения, анализа вывода программ и тестирования клиент-серверных приложений без участия пользователя.

4.2 Протоколы тестирования

Написанный сценарий `run_tests.exp` приведен в приложении [Д](#). Он автоматически запускает сервер и клиент, передаёт тестовые данные, принимает вывод программ и фиксирует результаты выполнения тестов.

В ходе тестирования проверены следующие случаи для подключений по TCP и UDP:

- корректный расчёт минимального пути в неориентированном графе;
- обработка ручного ввода графа с клавиатуры;
- обработка загрузки графа из файла;
- обработка случаев передачи графа, не соответствующего граничным условиям;
- корректность сетевого взаимодействия между клиентом и сервером;

Также произведено тестирование устойчивости приложения при передаче данных по протоколу UDP.

Вывод тестирующего приложения приведён в приложении [Е](#).

Заключение

В результате выполнения данной работы была достигнута поставленная цель — разработано клиент-серверное приложение, предназначенное для вычисления длины минимального пути в неориентированном взвешенном графе.

В процессе выполнения работы были решены все поставленные задачи. Были получены базовые знания по разработке программ в операционной системе Linux, а также освоены основные инструменты сборки и запуска приложений. Получены теоретические знания и практические навыки реализации межпроцессного взаимодействия с использованием механизма сокетов TCP/IP, включая работу с протоколами TCP и UDP.

Также были разработаны автоматические тесты, производящее тестирование клиент-серверного приложения. Разработанные тесты включают проверку ввода графа с клавиатуры и из файла, обработки граничных значений графа, корректность расчета кратчайшего пути в графе, а также проверку работы приложения по протоколам TCP и UDP.

В рамках работы были изучены принципы представления и обработки графов, а также реализован алгоритм Беллмана–Форда для поиска кратчайшего пути. Разработанное приложение корректно обрабатывает входные данные, выполняет вычисления на серверной стороне и передаёт результаты клиенту по сети.

Приложение А. «Исходный код клиентской части приложения»

Листинг 1 Исходный код client.cpp

```
1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <sstream>
5  #include <thread>
6  #include <chrono>
7  #include <cstring>
8  #include "graph.h"
9  #include "network.h"
10 #include "socket_utils.h"
11 #include "input_handler.h"
12
13 // класс клиентской части приложения
14 class GraphClient {
15 private:
16     std::string server_ip;
17     int server_port;
18     std::string protocol; // "TCP" или "UDP"
19
20 public:
21     GraphClient(const std::string& ip, int port, const std::string& proto)
22         : server_ip(ip), server_port(port), protocol(proto) {}
23
24     void start() {
25         std::cout << "=== Клиент поиска кратчайшего пути в графе"
26             << " ===" << std::endl;
27         std::cout << "Сервер: " << server_ip << ":" <<
28             server_port << std::endl;
29         std::cout << "Протокол: " << protocol << std::endl;
30
31         if (protocol == "TCP") {
32             startTCPClient();
33         } else if (protocol == "UDP") {
34             startUDPClient();
35         } else {
36             std::cerr << "Неизвестный протокол: " << protocol <<
37                 std::endl;
38             std::cerr << "Используйте TCP или UDP" << std::endl;
39         }
40     }
41
42 private:
43     void startTCPClient() {
44         int sockfd = -1;
```

```

43     try {
44         sockfd = SocketUtils::createTCPSocket();
45         SocketUtils::connectToServer(sockfd, server_ip,
46             server_port);
47         std::cout << "Успешно подключен к серверу по TCP" <<
48             std::endl;
49
50         mainLoop(sockfd);
51
52     } catch (const SocketException& e) {
53         std::cerr << "Ошибка подключения: " << e.what() <<
54             std::endl;
55         std::cerr << "Проверьте:" << std::endl;
56         std::cerr << "    - Запущен ли сервер на порту " <<
57             server_port << std::endl;
58         std::cerr << "    - Правильность IP-адреса" << std::
59             endl;
60     }
61
62     if (sockfd >= 0) {
63         SocketUtils::closeSocket(sockfd);
64     }
65 }
66
67 void startUDPClient() {
68     int sockfd = -1;
69
70     try {
71         sockfd = SocketUtils::createUDPSocket();
72         std::cout << "UDP клиент запущен" << std::endl;
73
74         mainUDPLoop(sockfd);
75
76     } catch (const SocketException& e) {
77         std::cerr << "Ошибка UDP клиента: " << e.what() <<
78             std::endl;
79     }
80
81     if (sockfd >= 0) {
82         SocketUtils::closeSocket(sockfd);
83     }
84 }
85
86 // меню help
87 void mainLoop(int sockfd) {
88     std::string command;
89
90     std::cout << "\nВведите 'help' для просмотра доступных ко
91         манд" << std::endl;
92
93     while (true) {
94         std::cout << "\nВведите команду: ";

```

```

88         std::getline(std::cin, command);
89
90         if (command == "exit") {
91             std::cout << "Завершение работы клиента..." <<
92                 std::endl;
93             try {
94                 SocketUtils::sendData(sockfd, "EXIT");
95             } catch (...) {
96                 // игнорируем ошибки при завершении
97             }
98             break;
99         } else if (command == "help") {
100             InputHandler::printHelp();
101         } else if (command == "input") {
102             handleManualInput(sockfd);
103         } else if (command == "file") {
104             handleFileInput(sockfd);
105         } else if (command == "test") {
106             sendTestRequest(sockfd);
107         } else {
108             std::cout << "Неизвестная команда. Введите 'help'
109                 для справки." << std::endl;
110         }
111     }
112
113     void mainUDPLoop(int sockfd) {
114         std::string command;
115
116         std::cout << "\nВведите 'help' для просмотра доступных ко
117             манд" << std::endl;
118
119         while (true) {
120             std::cout << "\nВведите команду: ";
121             std::getline(std::cin, command);
122
123             if (command == "exit") {
124                 std::cout << "Завершение работы клиента..." <<
125                     std::endl;
126                 break;
127             } else if (command == "help") {
128                 InputHandler::printHelp();
129             } else if (command == "input") {
130                 handleManualUDPInput(sockfd);
131             } else if (command == "file") {
132                 handleFileUDPInput(sockfd);
133             } else if (command == "test") {
134                 sendTestUDPRequest(sockfd);
135             } else {
136                 std::cout << "Неизвестная команда. Введите 'help'
137                     для справки." << std::endl;
138             }
139         }
140     }

```



```

135     }
136 }
137
138 // ручной ввод графа для TCP
139 void handleManualInput(int sockfd) {
140     try {
141         Request request = InputHandler::getUserInput();
142
143         if (InputHandler::validateInput(request)) {
144             sendTCPRequest(sockfd, request);
145         } else {
146             std::cout << "Данные не прошли проверку. Попробуй
147                             те снова." << std::endl;
148         }
149     } catch (const std::exception& e) {
150         std::cerr << "Ошибка ввода: " << e.what() << std::
151             endl;
152     }
153 }
154
155 // ручной ввод графа для UDP с надежной доставкой
156 void handleManualUDPInput(int sockfd) {
157     try {
158         Request request = InputHandler::getUserInput();
159
160         if (InputHandler::validateInput(request)) {
161             sendUDPRequestWithReliability(sockfd, request);
162         } else {
163             std::cout << "Данные не прошли проверку. Попробуй
164                             те снова." << std::endl;
165         }
166     } catch (const std::exception& e) {
167         std::cerr << "Ошибка ввода: " << e.what() << std::
168             endl;
169     }
170 }
171
172 // работа с файлом для TCP
173 void handleFileInput(int sockfd) {
174     std::cout << "Введите имя файла: ";
175     std::string filename;
176     std::getline(std::cin, filename);
177
178     try {
179         Request request = InputHandler::loadFromFile(filename
180             );
181
182         if (InputHandler::validateInput(request)) {
183             sendTCPRequest(sockfd, request);
184         } else {

```

```

182         std::cout << "Данные из файла не прошли проверку.
           " << std::endl;
183     }
184
185     } catch (const std::exception& e) {
186         std::cerr << "Ошибка загрузки файла: " << e.what() <<
           std::endl;
187     }
188 }
189
190 // работа с файлом для UDP
191 void handleFileUDPInput(int sockfd) {
192     std::cout << "Введите имя файла: ";
193     std::string filename;
194     std::getline(std::cin, filename);
195
196     try {
197         Request request = InputHandler::loadFromFile(filename
           );
198
199         if (InputHandler::validateInput(request)) {
200             sendUDPRequestWithReliability(sockfd, request);
201         } else {
202             std::cout << "Данные из файла не прошли проверку.
           " << std::endl;
203         }
204
205     } catch (const std::exception& e) {
206         std::cerr << "Ошибка загрузки файла: " << e.what() <<
           std::endl;
207     }
208 }
209
210 // тестовый запрос серверу для TCP
211 void sendTestRequest(int sockfd) {
212     Request request;
213     request.vertices = 6;
214     request.edges = {
215         {1, 2, 7}, {1, 3, 9}, {2, 3, 10}, {2, 4, 15},
216         {3, 4, 11}, {3, 6, 2}, {4, 5, 6}, {5, 6, 9}
217     };
218     request.start_vertex = 1;
219     request.end_vertex = 5;
220
221     sendTCPRequest(sockfd, request);
222 }
223
224 // тестовый запрос для UDP
225 void sendTestUDPRequest(int sockfd) {
226     Request request;
227     request.vertices = 6;
228     request.edges = {

```

```

229         {1, 2, 7}, {1, 3, 9}, {2, 3, 10}, {2, 4, 15},
230         {3, 4, 11}, {3, 6, 2}, {4, 5, 6}, {5, 6, 9}
231     };
232     request.start_vertex = 1;
233     request.end_vertex = 5;
234
235     sendUDPRequestWithReliability(sockfd, request);
236 }
237
238 // отправка TCP запроса
239 void sendTCPRequest(int sockfd, const Request& request) {
240     try {
241         std::string request_data = request.serialize();
242         SocketUtils::sendData(sockfd, request_data);
243         std::cout << "Запрос отправлен на сервер" << std::
                endl;
244
245         std::string response_data = SocketUtils::receiveData(
                sockfd);
246         Response response = Response::deserialize(
                response_data);
247
248         displayResponse(response);
249
250     } catch (const SocketException& e) {
251         if (std::string(e.what()) == "Соединение закрыто") {
252             std::cerr << "Сервер закрыл соединение. Перезапус
                тите клиент." << std::endl;
253         } else {
254             std::cerr << "Ошибка при отправке запроса: " << e
                .what() << std::endl;
255         }
256     } catch (const std::exception& e) {
257         std::cerr << "Ошибка: " << e.what() << std::endl;
258     }
259 }
260
261 // отправка UDP запроса с механизмом надежной доставки
262 // отправка UDP запроса с механизмом надежной доставки
263 void sendUDPRequestWithReliability(int sockfd, const Request&
request) {
264     std::string request_data = request.serialize();
265     std::string ack_msg = "ACK";
266     int max_attempts = 3;
267     int attempt = 0;
268     // структура адреса сервера для UDP
269     struct sockaddr_in server_addr;
270     memset(&server_addr, 0, sizeof(server_addr));
271     server_addr.sin_family = AF_INET;
272     server_addr.sin_port = htons(server_port);
273     if (inet_pton(AF_INET, server_ip.c_str(), &server_addr.
                sin_addr) <= 0) {

```

```

274         std::cerr << "Неверный IP-адрес: " << server_ip << std::
            endl;
275         return;
276     }
277     std::cout << "Отправка данных на сервер..." << std::endl;
278     while (attempt < max_attempts) {
279         try {
280             attempt++;
281             std::cout << "Попытка " << attempt << " из " <<
                max_attempts << std::endl;
282             // отправляем запрос
283             SocketUtils::sendTo(sockfd, request_data, server_addr
                );
284             std::cout << "Данные отправлены" << std::endl;
285             // ждем подтверждение в течение 3 секунд
286             struct sockaddr_in from_addr;
287             std::string ack_response;
288             try {
289                 ack_response = SocketUtils::receiveFrom(sockfd,
                    from_addr, 3);
290             } catch (const SocketException& e) {
291                 if (std::string(e.what()) == "Таймаут приема данн
                    ых") {
292                     std::cout << "Подтверждение не получено в теч
                        ение 3 секунд" << std::endl;
293                     if (attempt < max_attempts) {
294                         std::cout << "Повторная отправка..." <<
                            std::endl;
295                         std::this_thread::sleep_for(std::chrono::
                            milliseconds(500));
296                     }
297                     continue;
298                 } else {
299                     throw;
300                 }
301             }
302             if (ack_response == ack_msg) {
303                 std::cout << "Подтверждение получено" << std::
                    endl;
304                 // ждем окончательный ответ от сервера
305                 std::string response_data = SocketUtils::
                    receiveFrom(sockfd, from_addr, 5);
306                 Response response = Response::deserialize(
                    response_data);
307                 displayResponse(response);
308                 return;
309             } else {
310                 std::cout << "Получен неожиданный ответ: " <<
                    ack_response << std::endl;
311             }
312         } catch (const SocketException& e) {

```

```

313         if (std::string(e.what()) != "Таймаут приема данных")
314         {
315             std::cerr << "Ошибка: " << e.what() << std::endl;
316             break;
317         }
318     }
319     std::cout << "Потеряна связь с сервером" << std::endl;
320     std::cout << "Завершение работы клиента..." << std::endl;
321     exit(1);
322 }
323
324 // вывод результатов от сервера
325 void displayResponse(const Response& response) {
326     std::cout << "\n== Результат от сервера ==> " << std::
327         endl;
328     if (response.success) {
329         std::cout << "Успех! Кратчайший путь: " << response.
330             path_length << std::endl;
331         std::cout << "Маршрут: ";
332         for (size_t i = 0; i < response.path_vertices.size();
333             ++i) {
334             std::cout << response.path_vertices[i];
335             if (i < response.path_vertices.size() - 1) {
336                 std::cout << " -> ";
337             }
338         }
339         std::cout << std::endl;
340     } else {
341         std::cout << "Ошибка: " << response.error_message <<
342             std::endl;
343     }
344 }
345
346 int main(int argc, char* argv[]) {
347     if (argc < 4) {
348         std::cout << "Использование: " << argv[0] << " <IP> <PORT>
349             <> <PROTOCOL>" << std::endl;
350         std::cout << "Пример: " << argv[0] << " 127.0.0.1 8080
351             TCP" << std::endl;
352         std::cout << "Пример: " << argv[0] << " 127.0.0.1 8080
353             UDP" << std::endl;
354         return 1;
355     }
356
357     std::string ip = argv[1];
358     int port;
359     try {
360         port = std::stoi(argv[2]);
361     } catch (const std::exception& e) {

```

```

357         std::cerr << "Неверный номер порта: " << argv[2] << std::
            endl;
358         return 1;
359     }
360
361     std::string protocol = argv[3];
362
363     // проверка протокола
364     if (protocol != "TCP" && protocol != "UDP") {
365         std::cerr << "Неверный протокол. Используйте TCP или UDP"
            << std::endl;
366         return 1;
367     }
368
369     // проверка порта
370     if (port < 1 || port > 65535) {
371         std::cerr << "Неверный номер порта. Должен быть в диапазо
            не 1-65535" << std::endl;
372         return 1;
373     }
374
375     try {
376         GraphClient client(ip, port, protocol);
377         client.start();
378     } catch (const std::exception& e) {
379         std::cerr << "Фатальная ошибка: " << e.what() << std::
            endl;
380         return 1;
381     }
382
383     return 0;
384 }

```

Приложение Б. «Исходный код серверной части приложения»

Листинг 2 Исходный код server.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <signal.h>
4  #include <sys/select.h>
5  #include <thread>
6  #include <atomic>
7  #include <fcntl.h>
8  #include <map>
9  #include <chrono>
10 #include "graph.h"
11 #include "network.h"
12 #include "socket_utils.h"
13 #include <arpa/inet.h>
14
15 // класс серверной части приложения
16 class GraphServer {
17 private:
18     int server_port;
19     std::atomic<bool> running;
20     int tcp_socket;
21     int udp_socket;
22
23 public:
24     GraphServer(int port) : server_port(port), running(false),
25                             tcp_socket(-1), udp_socket(-1) {}
26
27     void start() {
28         std::cout << "=== Сервер поиска кратчайшего пути в графе"
29                     << std::endl;
30         std::cout << "Порт: " << server_port << std::endl;
31         std::cout << "Поддерживаемые протоколы: TCP и UDP" << std
32                     << std::endl;
33
34         running = true;
35
36         try {
37             // создаем TCP сокет
38             tcp_socket = SocketUtils::createTCPSocket();
39
40             // создаем UDP сокет
41             udp_socket = SocketUtils::createUDPSocket();
42
43             // устанавливаем неблокирующий режим для TCP сокета
44             int flags = fcntl(tcp_socket, F_GETFL, 0);
45             fcntl(tcp_socket, F_SETFL, flags | O_NONBLOCK);
```

```

44     SocketUtils::bindSocket(tcp_socket, server_port);
45     SocketUtils::bindSocket(udp_socket, server_port);
46     SocketUtils::listenSocket(tcp_socket, 5);
47
48     std::cout << "TCP и UDP сервер запущен и ожидает подк
        лючений..." << std::endl;
49     std::cout << "Для остановки сервера нажмите Ctrl+C"
        << std::endl;
50
51     mainLoop();
52
53     } catch (const SocketException& e) {
54         std::cerr << "Ошибка сервера: " << e.what() << std::
            endl;
55     }
56
57     cleanup();
58 }
59
60 void stop() {
61     running = false;
62 }
63
64 bool isRunning() const {
65     return running;
66 }
67
68 private:
69     void mainLoop() {
70         std::thread udp_thread(&GraphServer::handleUDPRequests,
            this);
71
72         while (running) {
73             try {
74                 struct sockaddr_in client_addr;
75
76                 // пытаемся принять TCP соединение с таймаутом
77                 int client_socket = acceptWithTimeout();
78
79                 if (client_socket >= 0) {
80                     // создаем отдельный поток для каждого клиент
                        а
81                     std::thread client_thread(&GraphServer::
                        handleTCPClient, this, client_socket,
                        client_addr);
82                     client_thread.detach();
83                 }
84
85             } catch (const SocketException& e) {
86                 if (running) {
87                     std::cerr << "Ошибка принятия соединения: "
                        << e.what() << std::endl;

```



```

88         }
89     }
90
91     // небольшая пауза чтобы не грузить CPU
92     std::this_thread::sleep_for(std::chrono::milliseconds
93         (100));
94
95     }
96
97     udp_thread.join();
98
99     void handleUDPRequests() {
100         std::cout << "UDP обработчик запущен" << std::endl;
101
102         while (running) {
103             try {
104                 struct sockaddr_in client_addr;
105
106                 // пытаемся получить UDP запрос с таймаутом
107                 fd_set read_fds;
108                 struct timeval timeout;
109
110                 FD_ZERO(&read_fds);
111                 FD_SET(udp_socket, &read_fds);
112
113                 timeout.tv_sec = 1;
114                 timeout.tv_usec = 0;
115
116                 int activity = select(udp_socket + 1, &read_fds,
117                     NULL, NULL, &timeout);
118
119                 if (activity < 0 && errno != EINTR) {
120                     throw SocketException("Ошибка select в UDP");
121                 }
122
123                 if (activity > 0 && FD_ISSET(udp_socket, &
124                     read_fds)) {
125                     std::string request_data = SocketUtils::
126                         receiveFrom(udp_socket, client_addr, 1);
127
128                     // проверяем, не служебное ли это сообщение
129                     if (request_data == "ACK") {
130                         // игнорируем ACK от клиента
131                         continue;
132                     }
133
134                     // логируем подключение
135                     char client_ip[INET_ADDRSTRLEN];
136                     if (inet_ntop(AF_INET, &(client_addr.sin_addr
137                         ), client_ip, INET_ADDRSTRLEN)) {
138                         std::cout << "- UDP запрос от клиента: "
139                             << client_ip << std::endl;

```

```

134         }
135
136         // отправляем подтверждение получения
137         SocketUtils::sendTo(udp_socket, "ACK",
138                             client_addr);
139
140         // обрабатываем запрос в отдельном потоке
141         std::thread udp_client_thread(&GraphServer::
142             handleUDPClient, this, request_data,
143             client_addr);
144         udp_client_thread.detach();
145     }
146
147     } catch (const SocketException& e) {
148         if (running && std::string(e.what()) != "Таймаут
149             приема данных") {
150             std::cerr << "Ошибка UDP обработки: " << e.
151                 what() << std::endl;
152         }
153     }
154
155     std::cout << "UDP обработчик остановлен" << std::endl;
156 }
157
158 int acceptWithTimeout() {
159     fd_set read_fds;
160     struct timeval timeout;
161
162     FD_ZERO(&read_fds);
163     FD_SET(tcp_socket, &read_fds);
164
165     // таймаут 1 секунда для возможности проверки running
166     timeout.tv_sec = 1;
167     timeout.tv_usec = 0;
168
169     int activity = select(tcp_socket + 1, &read_fds, NULL,
170         NULL, &timeout);
171
172     if (activity < 0 && errno != EINTR) {
173         throw SocketException("Ошибка select");
174     }
175
176     if (activity > 0 && FD_ISSET(tcp_socket, &read_fds)) {
177         struct sockaddr_in client_addr;
178         socklen_t client_len = sizeof(client_addr);
179
180         int client_socket = accept(tcp_socket, (struct
181             sockaddr*)&client_addr, &client_len);
182         if (client_socket >= 0) {
183             char client_ip[INET_ADDRSTRLEN];

```

```

178         const char* result = inet_ntop(AF_INET, &(
179             client_addr.sin_addr), client_ip,
180             INET_ADDRSTRLEN);
181
182         if (result != nullptr) {
183             std::cout << "- Новый TCP клиент подключен: "
184                 << client_ip << std::endl;
185         } else {
186             std::cout << "- Новый TCP клиент подключен (
187                 IP неизвестен)" << std::endl;
188         }
189
190         return client_socket;
191     }
192 }
193
194 return -1; // нет новых подключений
195 }
196
197 void handleTCPClient(int client_socket, const struct
198     sockaddr_in& client_addr) {
199     (void)client_addr; // параметр не используется в текущей
200     реализации
201
202     try {
203         // основной цикл обработки запросов от одного клиента
204         while (running) {
205             // получаем запрос от клиента
206             std::string request_data;
207             try {
208                 request_data = SocketUtils::receiveData(
209                     client_socket, 1); // таймаут 1 сек
210             } catch (const SocketException& e) {
211                 if (std::string(e.what()) == "Соединение закрыто") {
212                     std::cout << "TCP клиент отключился" <<
213                         std::endl;
214                     break;
215                 } else if (std::string(e.what()) == "Таймаут
216                     приема данных") {
217                     // продолжаем цикл и проверяем running
218                     continue;
219                 }
220                 throw;
221             }
222
223             // проверяем команду завершения
224             if (request_data == "EXIT") {
225                 std::cout << "TCP клиент запросил отключение"
226                     << std::endl;
227                 break;
228             }

```

```

219         Request request = Request::deserialize(
220             request_data);
221
222         std::cout << "TCP запрос получен: " << request.
223             vertices << " вершин, "
224             << request.edges.size() << " рёбер" <<
225             std::endl;
226
227         // обрабатываем запрос
228         Response response = processRequest(request);
229
230         // отправляем ответ
231         std::string response_data = response.serialize();
232         SocketUtils::sendData(client_socket,
233             response_data);
234
235         std::cout << "TCP ответ отправлен клиенту" << std
236             ::endl;
237     }
238
239 } catch (const std::exception& e) {
240     if (running) {
241         std::cerr << "Ошибка обработки TCP запроса: " <<
242             e.what() << std::endl;
243     }
244
245     // отправляем сообщение об ошибке
246     Response error_response;
247     error_response.success = false;
248     error_response.error_message = "Ошибка сервера: " +
249         std::string(e.what());
250
251     try {
252         std::string error_data = error_response.serialize
253             ();
254         SocketUtils::sendData(client_socket, error_data);
255     } catch (...) {
256         // игнорируем ошибки отправки
257     }
258
259     SocketUtils::closeSocket(client_socket);
260     std::cout << "TCP соединение с клиентом закрыто" << std::
261         endl;
262 }
263
264 void handleUDPClient(const std::string& request_data, const
265     struct sockaddr_in& client_addr) {
266     try {
267         Request request = Request::deserialize(request_data);
268

```

```

261         std::cout << "UDP запрос получен: " << request.
            vertices << " вершин, "
262             << request.edges.size() << " рёбер" << std
                ::endl;

263
264         // обрабатываем запрос
265         Response response = processRequest(request);
266
267         // отправляем ответ
268         std::string response_data = response.serialize();
269         SocketUtils::sendTo(udp_socket, response_data,
            client_addr);

270
271         std::cout << "UDP ответ отправлен клиенту" << std::
            endl;

272
273     } catch (const std::exception& e) {
274         std::cerr << "Ошибка обработки UDP запроса: " << e.
            what() << std::endl;

275
276         // пытаемся отправить сообщение об ошибке
277         try {
278             Response error_response;
279             error_response.success = false;
280             error_response.error_message = "Ошибка сервера: "
                + std::string(e.what());
281             std::string error_data = error_response.serialize
                ();
282             SocketUtils::sendTo(udp_socket, error_data,
                client_addr);
283         } catch (...) {
284             // игнорируем ошибки отправки
285         }
286     }
287 }

288
289 Response processRequest(const Request& request) {
290     Response response;
291
292     try {
293         // проверяем корректность данных
294         if (request.vertices < MIN_VERTICES || request.
            vertices > MAX_VERTICES) {
295             response.success = false;
296             response.error_message = "Некорректное количество
                вершин: " + std::to_string(request.vertices);
297             return response;
298         }
299
300         if (request.edges.size() < 6) {
301             response.success = false;

```

```

302         response.error_message = "Недостаточно рёбер: " +
303             std::to_string(request.edges.size()) + " (мин
304             имум 6)";
305         return response;
306     }
307
308     // создаем граф
309     Graph graph(request.vertices);
310     for (const auto& edge : request.edges) {
311         int from = std::get<0>(edge);
312         int to = std::get<1>(edge);
313         int weight = std::get<2>(edge);
314
315         if (weight <= 0) {
316             response.success = false;
317             response.error_message = "Некорректный вес ре
318             бпа: " + std::to_string(weight);
319             return response;
320         }
321
322         graph.addEdge(from, to, weight);
323     }
324
325     // ищем кратчайший путь
326     auto result = graph.findShortestPath(request.
327         start_vertex, request.end_vertex);
328
329     if (result.first > 0) {
330         response.success = true;
331         response.path_length = result.first;
332         response.path_vertices = result.second;
333         response.error_message = "";
334     } else {
335         response.success = false;
336         response.error_message = "Путь не найден между ве
337         ршинами " +
338             std::to_string(request.start_vertex) + " и "
339             +
340             std::to_string(request.end_vertex);
341     }
342
343     } catch (const std::exception& e) {
344         response.success = false;
345         response.error_message = "Ошибка обработки: " + std::
346             string(e.what());
347     }
348
349     return response;
350 }
351
352 void cleanup() {
353     if (tcp_socket >= 0) {

```

```

347         SocketUtils::closeSocket(tcp_socket);
348     }
349     if (udp_socket >= 0) {
350         SocketUtils::closeSocket(udp_socket);
351     }
352     std::cout << "Сервер остановлен" << std::endl;
353 }
354 };
355
356 // переменная для обработки сигналов
357 GraphServer* server_ptr = nullptr;
358
359 void signalHandler(int signal) {
360     std::cout << "\nПолучен сигнал " << signal << ", останавливаем сервер..." << std::endl;
361     if (server_ptr) {
362         server_ptr->stop();
363     }
364 }
365
366 int main(int argc, char* argv[]) {
367     int port = 8080; // по умолчанию
368
369     if (argc > 1) {
370         port = std::stoi(argv[1]);
371     }
372
373     // обработчики сигналов
374     signal(SIGINT, signalHandler);
375     signal(SIGTERM, signalHandler);
376
377     GraphServer server(port);
378     server_ptr = &server;
379
380     server.start();
381
382     return 0;
383 }

```

Приложение В. «Исходный код заголовочных файлов»

Листинг 3 Исходный код graph.h

```
1 #ifndef GRAPH_H
2 #define GRAPH_H
3
4 #include <vector>
5 #include <iostream>
6 #include <limits>
7 #include <string>
8
9 static const int MAX_VERTICES = 100;
10 static const int MIN_VERTICES = 6;
11 static const int INF = std::numeric_limits<int>::max();
12
13 // структура для описания графа
14 class Graph {
15 private:
16     int vertices;
17
18             // количество вершин
19     std::vector<std::vector<std::pair<int, int>>> adjacencyList;
20             // список смежности
21
22 public:
23     Graph();
24     Graph(int vertices);
25
26     void addEdge(int from, int to, int weight);
27     bool validateGraph() const;
28     int getVerticesCount() const;
29
30     std::pair<int, std::vector<int>> findShortestPath(int start,
31     int end) const; // реализация алгоритма Беллмана-Форда
32
33     const std::vector<std::vector<std::pair<int, int>>>&
34     getAdjacencyList() const;
35     void printGraph() const;
36 };
37
38 #endif
```

Листинг 4 Исходный код network.h

```
1 #ifndef NETWORK_H
2 #define NETWORK_H
3
4 #include <vector>
5 #include <string>
```



```

6
7 // структура для запроса от клиента к серверу
8 struct Request {
9     int vertices;
10    std::vector<std::tuple<int, int, int>> edges;           // (
        from, to, weight)
11    int start_vertex;
12    int end_vertex;
13
14    std::string serialize() const;                         // сери
        ализация в строку
15
16    static Request deserialize(const std::string& data);    // десе
        риализация из строки
17 };
18
19 // структура для ответа от сервера клиенту
20 struct Response {
21     bool success;
22     int path_length;
23     std::vector<int> path_vertices;
24     std::string error_message;
25
26     std::string serialize() const;                         // се
        риализация в строку
27
28     static Response deserialize(const std::string& data);    // де
        сериализация из строки
29 };
30
31 #endif

```

Листинг 5 Исходный код socket_utils.h

```

1 #ifndef SOCKET_UTILS_H
2 #define SOCKET_UTILS_H
3
4 #include <string>
5 #include <sys/socket.h>
6 #include <netinet/in.h>
7 #include <arpa/inet.h>
8 #include <unistd.h>
9 #include <cstring>
10 #include <stdexcept>
11 #include <errno.h>
12
13 // класс исключений для ошибок сокетов
14 class SocketException : public std::runtime_error {
15 public:
16     SocketException(const std::string& message) : std::
        runtime_error(message) {}
17 };

```

```

18
19 // класс для утилит работы с сокетами
20 class SocketUtils {
21 public:
22     static int createTCPSocket();
23     static int createUDPSocket();
24     static void setNonBlocking(int socket_fd, bool nonblocking);
25     static void bindSocket(int socket_fd, int port);
26     static void listenSocket(int socket_fd, int backlog = 5);
27     static int acceptConnection(int socket_fd, struct sockaddr_in
        & client_addr);
28     static void connectToServer(int socket_fd, const std::string&
        ip, int port);
29     static void sendData(int socket_fd, const std::string& data);
30     static void sendTo(int socket_fd, const std::string& data,
        const struct sockaddr_in& addr);
31     static std::string receiveData(int socket_fd, int timeout_sec
        = 0);
32     static std::string receiveFrom(int socket_fd, struct
        sockaddr_in& from_addr, int timeout_sec = 0);
33     static void closeSocket(int socket_fd);
34 };
35
36 #endif

```

Листинг 6 Исходный код input_handler.h

```

1 #ifndef INPUT_HANDLER_H
2 #define INPUT_HANDLER_H
3
4 #include <string>
5 #include <vector>
6 #include "network.h"
7
8 // класс обработки пользовательского ввода
9 class InputHandler {
10 public:
11     static Request getUserInput();
12     static Request loadFromFile(const std::string& filename);
13     static bool validateInput(const Request& request);
14     static void printHelp();
15
16 private:
17     static int getIntegerInput(const std::string& prompt, int min
        , int max);
18     static std::vector<std::tuple<int, int, int>> parseEdges(
        const std::string& input, int vertex_count);
19     static bool isValidVertex(int vertex, int vertex_count);
20 };
21
22 #endif

```

Приложение Г. «Исходный код файлов реализации»

Листинг 7 Исходный код graph.cpp

```
1  #include "graph.h"
2  #include <queue>
3  #include <algorithm>
4
5  Graph::Graph() : vertices(0) {}
6
7  Graph::Graph(int vertices) : vertices(vertices) {
8      adjacencyList.resize(vertices + 1);
9  }
10
11 void Graph::addEdge(int from, int to, int weight) {
12     if (from > 0 && from <= vertices && to > 0 && to <= vertices
13         && weight > 0) {
14         adjacencyList[from].push_back({to, weight});
15         adjacencyList[to].push_back({from, weight});
16     }
17 }
18
19 bool Graph::validateGraph() const {
20     return vertices >= MIN_VERTICES && vertices <= MAX_VERTICES;
21 }
22
23 int Graph::getVerticesCount() const {
24     return vertices;
25 }
26
27 // алгоритм Беллмана-Форда
28 // этап проверки графа на наличие отрицательных циклов опущен, т.
29 // к. в рамках поставленной задачи в графе не может быть отрицате-
30 // льного цикла
31
32 std::pair<int, std::vector<int>> Graph::findShortestPath(int
33 start, int end) const {
34     if (start < 1 || start > vertices || end < 1 || end >
35         vertices) {
36         return {-1, {}};
37     }
38
39     std::vector<int> distance(vertices + 1, INF);
40     std::vector<int> predecessor(vertices + 1, -1);
41
42     distance[start] = 0;
43
44     for (int i = 1; i <= vertices - 1; ++i) {
45         for (int u = 1; u <= vertices; ++u) {
46             if (distance[u] == INF) continue;
```

```

42         for (const auto& edge : adjacencyList[u]) {
43             int v = edge.first;
44             int weight = edge.second;
45
46             if (distance[u] + weight < distance[v]) {
47                 distance[v] = distance[u] + weight;
48                 predecessor[v] = u;
49             }
50         }
51     }
52 }
53
54 if (distance[end] == INF) {
55     return {-1, {}};
56 }
57
58 std::vector<int> path;
59 for (int v = end; v != -1; v = predecessor[v]) {
60     path.push_back(v);
61 }
62 std::reverse(path.begin(), path.end());
63
64 return {distance[end], path};
65 }
66
67 const std::vector<std::vector<std::pair<int, int>>>& Graph::
68     getAdjacencyList() const {
69     return adjacencyList;
70 }
71
72 void Graph::printGraph() const {
73     std::cout << "Граф с " << vertices << " вершинами:" << std::
74         endl;
75     for (int i = 1; i <= vertices; ++i) {
76         std::cout << "Вершина " << i << ": ";
77         for (const auto& neighbor : adjacencyList[i]) {
78             std::cout << "-> " << neighbor.first << " (вес:" <<
79                 neighbor.second << ") ";
80         }
81         std::cout << std::endl;
82     }
83 }

```

Листинг 8 Исходный код network.cpp

```

1 #include "network.h"
2 #include <sstream>
3 #include <iostream>
4
5
6 // структура данных -> "vertices | start_vertex | end_vertex |
   from1, to1, weight1; from2, to2, weight2"

```

```

7 std::string Request::serialize() const {
8     std::stringstream ss;
9     ss << vertices << "|" << start_vertex << "|" << end_vertex <<
    " | ";
10
11     for (const auto& edge : edges) {
12         ss << std::get<0>(edge) << "," << std::get<1>(edge) << ",
            " << std::get<2>(edge) << ";";
13     }
14
15     return ss.str();
16 }
17
18 // разбор строки по по разделителям
19 // извлечение количества вершин, стартовой и конечной вершины, сн
    и ска ребер
20 Request Request::deserialize(const std::string& data) {
21     Request req;
22     std::stringstream ss(data);
23     char delimiter;
24
25     ss >> req.vertices >> delimiter;
26     ss >> req.start_vertex >> delimiter;
27     ss >> req.end_vertex >> delimiter;
28
29     std::string edges_str;
30     ss >> edges_str;
31
32     std::stringstream edges_ss(edges_str);
33     std::string edge_token;
34
35     while (std::getline(edges_ss, edge_token, ';')) {
36         if (edge_token.empty()) continue;
37
38         std::stringstream edge_ss(edge_token);
39         int from, to, weight;
40         char comma;
41
42         edge_ss >> from >> comma >> to >> comma >> weight;
43         req.edges.push_back({from, to, weight});
44     }
45
46     return req;
47 }
48
49 // успех: "1 | длина пути | вершины пути | "
50 // неуспех: "0 | 0 | | сообщение об ошибке "
51 std::string Response::serialize() const {
52     std::stringstream ss;
53     ss << success << "|" << path_length << " | ";
54
55     for (int vertex : path_vertices) {

```

```

56         ss << vertex << ",";
57     }
58
59     ss << "|" << error_message;
60     return ss.str();
61 }
62
63 // извлечение длины пути и списка ребер или сообщения об ошибке
64 Response Response::deserialize(const std::string& data) {
65     Response resp;
66     std::stringstream ss(data);
67     char delimiter;
68
69     ss >> resp.success >> delimiter;
70     ss >> resp.path_length >> delimiter;
71
72     std::string path_str;
73     std::getline(ss, path_str, '|');
74
75     std::stringstream path_ss(path_str);
76     std::string vertex_token;
77
78     while (std::getline(path_ss, vertex_token, ',')) {
79         if (vertex_token.empty()) continue;
80         resp.path_vertices.push_back(std::stoi(vertex_token));
81     }
82
83     std::getline(ss, resp.error_message);
84     return resp;
85 }

```

Листинг 9 Исходный код socket_utils.cpp

```

1  #include "socket_utils.h"
2  #include <iostream>
3  #include <fcntl.h>
4
5  // создание сокетов
6  int SocketUtils::createTCPSocket() {
7      int sockfd = socket(AF_INET, SOCK_STREAM, 0);
8      if (sockfd < 0) {
9          throw SocketException("Ошибка создания TCP сокета");
10     }
11     return sockfd;
12 }
13
14 int SocketUtils::createUDPSocket() {
15     int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
16     if (sockfd < 0) {
17         throw SocketException("Ошибка создания UDP сокета");
18     }
19     return sockfd;

```

```

20 }
21
22 // перевести сокет в неблокирующий режим
23 void SocketUtils::setNonBlocking(int socket_fd, bool nonblocking)
24 {
25     int flags = fcntl(socket_fd, F_GETFL, 0);
26     if (flags < 0) {
27         throw SocketException("Ошибка получения флагов сокета");
28     }
29
30     if (nonblocking) {
31         flags |= O_NONBLOCK;
32     } else {
33         flags &= ~O_NONBLOCK;
34     }
35
36     if (fcntl(socket_fd, F_SETFL, flags) < 0) {
37         throw SocketException("Ошибка установки флагов сокета");
38     }
39 }
40
41 // ==== настройка сервера ====
42 // привязка сокета к конкретному порту
43 void SocketUtils::bindSocket(int socket_fd, int port) {
44     struct sockaddr_in server_addr;
45     memset(&server_addr, 0, sizeof(server_addr));
46     server_addr.sin_family = AF_INET;
47     server_addr.sin_addr.s_addr = INADDR_ANY;
48     server_addr.sin_port = htons(port);
49
50     if (bind(socket_fd, (struct sockaddr*)&server_addr, sizeof(
51         server_addr)) < 0) {
52         throw SocketException("Ошибка привязки сокета к порту " +
53             std::to_string(port));
54     }
55 }
56
57 // перевод сокета в режим прослушивания
58 void SocketUtils::listenSocket(int socket_fd, int backlog) { //
59     backlog - максимальная длина очереди ожидающих соединений
60     if (listen(socket_fd, backlog) < 0) {
61         throw SocketException("Ошибка прослушивания сокета");
62     }
63 }
64
65 // принятие входящего соединения
66 int SocketUtils::acceptConnection(int socket_fd, struct
67     sockaddr_in& client_addr) {
68     socklen_t client_len = sizeof(client_addr);
69     int client_sockfd = accept(socket_fd, (struct sockaddr*)&
70         client_addr, &client_len);
71     if (client_sockfd < 0) {

```

```

66         throw SocketException("Ошибка принятия соединения");
67     }
68     return client_sockfd;
69 }
70
71 // ==== настройка клиента ====
72 // установка соединения с сервером
73 void SocketUtils::connectToServer(int socket_fd, const std::
    string& ip, int port) {
74     struct sockaddr_in server_addr;
75     memset(&server_addr, 0, sizeof(server_addr));
76     server_addr.sin_family = AF_INET;
77     server_addr.sin_port = htons(port);
78
79     if (inet_pton(AF_INET, ip.c_str(), &server_addr.sin_addr) <=
        0) {
80         throw SocketException("Неверный IP-адрес: " + ip);
81     }
82
83     if (connect(socket_fd, (struct sockaddr*)&server_addr, sizeof
        (server_addr)) < 0) {
84         throw SocketException("Ошибка подключения к серверу " +
            ip + ":" + std::to_string(port));
85     }
86 }
87
88 // передача данных
89 void SocketUtils::sendData(int socket_fd, const std::string& data
    ) {
90     size_t total_sent = 0;
91     size_t data_length = data.length();
92
93     while (total_sent < data_length) {
94         ssize_t sent = send(socket_fd, data.c_str() + total_sent,
            data_length - total_sent, 0);
95         if (sent < 0) {
96             throw SocketException("Ошибка отправки данных");
97         }
98         total_sent += static_cast<size_t>(sent);
99     }
100 }
101
102 // отправка данных по UDP с указанием адреса получателя
103 void SocketUtils::sendTo(int socket_fd, const std::string& data,
    const struct sockaddr_in& addr) {
104     ssize_t sent = sendto(socket_fd, data.c_str(), data.length(),
        0,
105                             (struct sockaddr*)&addr, sizeof(addr));
106     if (sent < 0) {
107         throw SocketException("Ошибка отправки данных по UDP");
108     }
109 }

```



```

110
111 // принятие данных с сервера
112 std::string SocketUtils::receiveData(int socket_fd, int
    timeout_sec) {
113     char buffer[4096]; // 4KB для приема
114     std::string result;
115
116     if (timeout_sec > 0) { // таймаут для операц
        ии приема
117         struct timeval tv;
118         tv.tv_sec = timeout_sec;
119         tv.tv_usec = 0;
120         setsockopt(socket_fd, SOL_SOCKET, SO_RCVTIMEO, &tv,
            sizeof(tv));
121     }
122
123     ssize_t bytes_received = recv(socket_fd, buffer, sizeof(
        buffer) - 1, 0);
124     if (bytes_received > 0) { // данные получены
125         buffer[bytes_received] = '\0';
126         result = buffer;
127     } else if (bytes_received == 0) {
128         throw SocketException("Соединение закрыто");
129     } else {
130         if (timeout_sec > 0 && (errno == EAGAIN || errno ==
            EWOULDBLOCK)) {
131             throw SocketException("Таймаут приема данных");
132         }
133         throw SocketException("Ошибка приема данных");
134     }
135
136     return result;
137 }
138
139 // прием данных по UDP с получением адреса отправителя
140 std::string SocketUtils::receiveFrom(int socket_fd, struct
    sockaddr_in& from_addr, int timeout_sec) {
141     char buffer[4096];
142     socklen_t addr_len = sizeof(from_addr);
143
144     if (timeout_sec > 0) {
145         struct timeval tv;
146         tv.tv_sec = timeout_sec;
147         tv.tv_usec = 0;
148         setsockopt(socket_fd, SOL_SOCKET, SO_RCVTIMEO, &tv,
            sizeof(tv));
149     }
150
151     ssize_t bytes_received = recvfrom(socket_fd, buffer, sizeof(
        buffer) - 1, 0,
152                                     (struct sockaddr*)&from_addr
        , &addr_len);

```

```

153
154     if (bytes_received > 0) {
155         buffer[bytes_received] = '\0';
156         return std::string(buffer);
157     } else if (bytes_received == 0) {
158         throw SocketException("Соединение закрыто");
159     } else {
160         if (timeout_sec > 0 && (errno == EAGAIN || errno ==
161             EWOULDBLOCK)) {
162             throw SocketException("Таймаут приема данных");
163         }
164         throw SocketException("Ошибка приема данных по UDP");
165     }
166 }
167
168 void SocketUtils::closeSocket(int socket_fd) {
169     close(socket_fd);
170 }

```

Листинг 10 Исходный код input_handler.cpp

```

1  #include "input_handler.h"
2  #include "graph.h"
3  #include <iostream>
4  #include <sstream>
5  #include <fstream>
6  #include <regex>
7
8  Request InputHandler::getUserInput() {
9      Request request;
10
11      std::cout << "\n=== Ввод данных о графе ===" << std::endl;
12
13      // запрашиваем количество вершин
14      request.vertices = getIntegerInput(
15          "Введите количество вершин в графе (целое число от 6 до
16          100): ",
17          MIN_VERTICES, MAX_VERTICES
18      );
19
20      // запрашиваем рёбра графа
21      std::cout << "Перечислите рёбра графа в формате (вершина1, ве
22      ршина2, вес):" << std::endl;
23      std::cout << "Пример: (1,2,5) (2,3,3) (1,3,10)" << std::endl;
24      std::cout << "Введите рёбра: ";
25
26      std::string edges_input;
27      std::getline(std::cin, edges_input);
28      request.edges = parseEdges(edges_input, request.vertices);
29
30      // запрашиваем начальную и конечную вершины
31      request.start_vertex = getIntegerInput(

```

```

30         "Введите индекс начальной вершины (от 1 до " + std::
          to_string(request.vertices) + "): ",
31         1, request.vertices
32     );
33
34     request.end_vertex = getIntegerInput(
35         "Введите индекс конечной вершины (от 1 до " + std::
          to_string(request.vertices) + "): ",
36         1, request.vertices
37     );
38
39     return request;
40 }
41
42 Request InputHandler::loadFromFile(const std::string& filename) {
43     Request request;
44     std::ifstream file(filename);
45
46     if (!file.is_open()) {
47         throw std::runtime_error("Не удалось открыть файл: " +
          filename);
48     }
49
50     std::string line;
51
52     // читаем количество вершин
53     if (std::getline(file, line)) {
54         request.vertices = std::stoi(line);
55     }
56
57     // читаем рёбра
58     if (std::getline(file, line)) {
59         request.edges = parseEdges(line, request.vertices);
60     }
61
62     // читаем начальную вершину
63     if (std::getline(file, line)) {
64         request.start_vertex = std::stoi(line);
65     }
66
67     // читаем конечную вершину
68     if (std::getline(file, line)) {
69         request.end_vertex = std::stoi(line);
70     }
71
72     file.close();
73     return request;
74 }
75
76 bool InputHandler::validateInput(const Request& request) {
77     if (request.vertices < MIN_VERTICES || request.vertices >
        MAX_VERTICES) {

```

```

78     std::cerr << "Ошибка: количество вершин должно быть от "
       << MIN_VERTICES
79         << " до " << MAX_VERTICES << std::endl;
80     return false;
81 }
82
83 if (request.edges.size() < 6) {
84     std::cerr << "Ошибка: количество рёбер должно быть не мен
       ee 6" << std::endl;
85     return false;
86 }
87
88 int max_edges = request.vertices * (request.vertices - 1) /
      2;
89 if (request.edges.size() > static_cast<size_t>(max_edges)) {
90     std::cerr << "Ошибка: слишком много рёбер для графа с "
       << request.vertices << " вершинами" << std::endl;
91     return false;
92 }
93
94 if (!isValidVertex(request.start_vertex, request.vertices)) {
95     std::cerr << "Ошибка: начальная вершина должна быть от 1
       до " << request.vertices << std::endl;
96     return false;
97 }
98
99 if (!isValidVertex(request.end_vertex, request.vertices)) {
100     std::cerr << "Ошибка: конечная вершина должна быть от 1 д
       о " << request.vertices << std::endl;
101     return false;
102 }
103
104 // проверка весов рёбер
105 for (const auto& edge : request.edges) {
106     int weight = std::get<2>(edge);
107     if (weight <= 0) {
108         std::cerr << "Ошибка: вес ребра должен быть положител
       ьным" << std::endl;
109         return false;
110     }
111 }
112
113 return true;
114 }
115
116 void InputHandler::printHelp() {
117     std::cout << "\n=== Справочник команд ===" << std::endl;
118     std::cout << "help      - показать эту справку" << std::endl;
119     std::cout << "input    - ввести данные о графе вручную" << std
       ::endl;
120     std::cout << "file      - загрузить данные из файла" << std::
       endl;

```

```

121     std::cout << "test      - отправить тестовый запрос" << std::
        endl;
122     std::cout << "exit      - выйти из программы" << std::endl;
123     std::cout << "\nФормат ввода рёбер: (1,2,5) (2,3,3) (1,3,10)"
        << std::endl;
124     std::cout << "Формат файла:" << std::endl;
125     std::cout << "    первая строка: количество вершин" << std::
        endl;
126     std::cout << "    вторая строка: рёбра в формате (1,2,5)
        (2,3,3)" << std::endl;
127     std::cout << "    третья строка: начальная вершина" << std::
        endl;
128     std::cout << "    четвертая строка: конечная вершина" << std::
        endl;
129 }
130
131 int InputHandler::getIntegerInput(const std::string& prompt, int
    min, int max) {
132     int value;
133     while (true) {
134         std::cout << prompt;
135         std::string input;
136         std::getline(std::cin, input);
137
138         try {
139             value = std::stoi(input);
140             if (value >= min && value <= max) {
141                 break;
142             } else {
143                 std::cout << "Ошибка: введите число от " << min
                    << " до " << max << std::endl;
144             }
145         } catch (const std::exception&) {
146             std::cout << "Ошибка: введите целое число" << std::
                endl;
147         }
148     }
149     return value;
150 }
151
152 std::vector<std::tuple<int, int, int>> InputHandler::parseEdges(
    const std::string& input, int vertex_count) {
153     std::vector<std::tuple<int, int, int>> edges;
154
155     // регулярное выражение для поиска рёбер в формате (a,b,w)
156     std::regex edge_regex(R"(\((\d+),\s*(\d+),\s*(\d+)\))");
157     std::smatch match;
158
159     std::string::const_iterator search_start(input.cbegin());
160
161     while (std::regex_search(search_start, input.cend(), match,
        edge_regex)) {

```

```

162     int from = std::stoi(match[1].str());
163     int to = std::stoi(match[2].str());
164     int weight = std::stoi(match[3].str());
165
166     if (isValidVertex(from, vertex_count) && isValidVertex(to
167         , vertex_count) && weight > 0) {
168         edges.push_back({from, to, weight});
169     } else {
170         std::cout << "Предупреждение: пропущено некорректное
171             ребро ("
172                 << from << "," << to << "," << weight << ")
173                 " << std::endl;
174     }
175
176     search_start = match.suffix().first;
177 }
178
179 if (edges.empty()) {
180     throw std::runtime_error("Не найдено корректных рёбер во
181         вводе");
182 }
183
184 return edges;
185 }
186
187 bool InputHandler::isValidVertex(int vertex, int vertex_count) {
188     return vertex >= 1 && vertex <= vertex_count;
189 }

```

Приложение Д. «Исходный код тестирующего приложения»

Листинг 11 Исходный код run_tests.exp

```
1  #!/usr/bin/expect -f
2
3  set server "./server"
4  set client "./client"
5  set ip "127.0.0.1"
6  set port "8080"
7  set timeout 15
8
9  # флаг успеха
10 set all_passed 1
11
12 # вспомогательная процедура для запуска теста
13 proc run_test {name script} {
14     global all_passed
15     puts "\nТест: $name"
16     if {[catch {eval $script} err]} {
17         puts "    FAIL: $err"
18         set all_passed 0
19     } else {
20         puts "    PASS"
21     }
22 }
23
24 # процедура: запустить сервер в фоне и дать ему время
25 proc start_server {} {
26     global server port
27     spawn $server $port
28     sleep 1
29     # сервер запущен в фоне; expect не будет ждать его завершения
30     set pid [exp_pid]
31     return $pid
32 }
33
34 # процедура: убить процесс по PID
35 proc kill_server {pid} {
36     exec kill -TERM $pid
37     catch {exec kill -KILL $pid} ; # на всякий
38 }
39
40 # процедура: создать временный файл с графом
41 proc create_test_file {filename vertices edges start end} {
42     set f [open $filename w]
43     puts $f $vertices
44     set edge_str ""
45     foreach edge $edges {
46         lassign $edge u v w
```

```

47         append edge_str "($u,$v,$w) "
48     }
49     puts $f [string trim $edge_str]
50     puts $f $start
51     puts $f $end
52     close $f
53 }
54
55 # === ТЕСТЫ ===
56
57 # TCP, ввод с клавиатуры, N=6
58 run_test "TCP: ввод с клавиатуры, N=6" {
59     set server_pid [start_server]
60     spawn $::client $::ip $::port TCP
61     expect "Введите команду:"
62     send "input\r"
63     expect "Введите количество вершин"
64     send "6\r"
65     expect "Перечислите рёбра"
66     send "(1,2,1) (2,3,1) (3,4,1) (4,5,1) (5,6,1) (6,1,1)\r"
67     expect "начальной вершины"
68     send "1\r"
69     expect "конечной вершины"
70     send "4\r"
71     expect {
72         -re "Успех.*" { }
73         -re "Ошибка.*" { error "Ожидался успех, получена ошибка"
74             }
75         timeout { error "Таймаут ожидания результата" }
76     }
77     send "exit\r"
78     kill_server $server_pid
79 }
80
81 # TCP, файл, N=53
82 run_test "TCP: ввод из файла, N=53" {
83     set server_pid [start_server]
84     set tmpfile "/tmp/test_graph_53.txt"
85     create_test_file $tmpfile 53 {
86         {1 2 1} {2 3 1} {3 4 1} {4 5 1} {5 6 1} {6 1 1}
87     } 1 5
88
89     spawn $::client $::ip $::port TCP
90     expect "Введите команду:"
91     send "file\r"
92     expect "Введите имя файла:"
93     send "$tmpfile\r"
94     expect {
95         -re "Успех.*" { }
96         -re "Ошибка.*" { error "Ожидался успех" }
97         timeout { error "Таймаут" }
98     }
99 }

```



```

98     send "exit\r"
99     kill_server $server_pid
100    file delete $tmpfile
101 }
102
103 # UDP, ввод с клавиатуры, N=100
104 run_test "UDP: ввод с клавиатуры, N=100" {
105     set server_pid [start_server]
106     spawn $::client $::ip $::port UDP
107     expect "Введите команду:"
108     send "input\r"
109     expect "Введите количество вершин"
110     send "100\r"
111     expect "Перечислите рёбра"
112     send "(1,2,1) (2,3,1) (3,4,1) (4,5,1) (5,6,1) (6,1,1)\r"
113     expect "начальной вершины"
114     send "1\r"
115     expect "конечной вершины"
116     send "2\r"
117     expect {
118         -re "Успех.*" { }
119         -re "Ошибка.*" { error "Ожидался успех" }
120         timeout { error "Таймаут" }
121     }
122     send "exit\r"
123     kill_server $server_pid
124 }
125
126 # N=5 (ниже ОДЗ) клиент должен отклонить
127 run_test "TCP: N=5 (недопустимо)" {
128     set server_pid [start_server]
129     spawn $::client $::ip $::port TCP
130     expect "Введите команду:"
131     send "input\r"
132     expect "Введите количество вершин"
133     send "5\r"
134     expect {
135         -re "Ошибка:.*" { }
136         -re "Перечислите рёбра" { error "Клиент не отклонил N=5" }
137         timeout { error "Таймаут" }
138     }
139     send "exit\r"
140     kill_server $server_pid
141 }
142
143 # N=101 (выше ОДЗ) клиент должен отклонить
144 run_test "TCP: N=101 (недопустимо)" {
145     set server_pid [start_server]
146     spawn $::client $::ip $::port TCP
147     expect "Введите команду:"
148     send "input\r"

```

```

149 expect "Введите количество вершин"
150 send "101\r"
151 expect {
152     -re "Ошибка:.*" { }
153     -re "Перечислите рёбра" { error "Клиент не отклонил N=101" }
154     timeout { error "Таймаут" }
155 }
156 send "exit\r"
157 kill_server $server_pid
158 }
159
160 # UDP, сервер недоступен
161 run_test "UDP: сервер недоступен" {
162     # НЕ запускаем сервер!
163     spawn $::client $::ip $::port UDP
164     expect "Введите команду:"
165     send "input\r"
166     expect "Введите количество вершин"
167     send "6\r"
168     expect "Перечислите рёбра"
169     send "(1,2,1) (2,3,1) (3,4,1) (4,5,1) (5,6,1) (6,1,1)\r"
170     expect "начальной вершины"
171     send "1\r"
172     expect "конечной вершины"
173     send "2\r"
174     expect {
175         -re "Потеряна связь с сервером" { }
176         -re "Успех.*" { error "Сервер не должен был ответить" }
177         -re "Попытка.*из.*" {
178             # Ждём, пока не закончатся попытки
179             exp_continue
180         }
181         timeout { error "Нет сообщения о потере связи" }
182     }
183     send "exit\r"
184 }
185
186 # корректность вычисленного кратчайшего пути (TCP)
187 run_test "TCP: проверка правильности кратчайшего пути (ожидаем пу
    ть длины 20: 1->3->6->5)" {
188     set server_pid [start_server]
189     spawn $::client $::ip $::port TCP
190     expect "Введите команду:"
191     send "input\r"
192     expect "Введите количество вершин"
193     send "6\r"
194     expect "Перечислите рёбра"
195     send "(1,2,7) (1,3,9) (2,3,10) (2,4,15) (3,4,11) (3,6,2)
        (4,5,6) (5,6,9)\r"
196     expect "начальной вершины"
197     send "1\r"

```

```

198     expect "конечной вершины"
199     send "5\r"
200     expect {
201         -re "Успех! Кратчайший путь: 30" {
202             # Проверяем маршрут
203             expect {
204                 -re "Маршрут: 1 -> 3 -> 6 -> 5" { }
205                 -re "Маршрут:.*" { error "Неверная последовательность вершин в пути" }
206                 timeout { error "Не найден маршрут" }
207             }
208         }
209         -re "Ошибка.*" { error "Запрос завершился ошибкой" }
210         timeout { error "Таймаут ожидания результата" }
211     }
212     send "exit\r"
213     kill_server $server_pid
214 }
215
216 # финальный отчёт
217 if {$all_passed} {
218     puts "\nВсе тесты пройдены успешно!"
219     exit 0
220 } else {
221     puts "\nНекоторые тесты провалены!"
222     exit 1
223 }

```

Приложение Е. «Протоколы тестирования»

На рис. 3 приведен результат теста соединения по протоколу TCP с вводом графа с клавиатуры. Введенный граф содержит 6 вершин и 6 ребер.

```
punshhii@punshhii-VirtualBox:/media/sf_code/clientServerGraph/build$ expect ./run_tests.exp

Тест: TCP: ввод с клавиатуры, N=6
spawn ./server 8080
spawn ./client 127.0.0.1 8080 TCP
=== Клиент поиска кратчайшего пути в графе ===
Сервер: 127.0.0.1:8080
Протокол: TCP
Успешно подключен к серверу по TCP

Введите 'help' для просмотра доступных команд

Введите команду: input

=== Ввод данных о графе ===
Введите количество вершин в графе (целое число от 6 до 100): 6
Перечислите рѐбра графа в формате (вершина1, вершина2, вес):
Пример: (1,2,5) (2,3,3) (1,3,10)
Введите рѐбра: (1,2,1) (2,3,1) (3,4,1) (4,5,1) (5,6,1) (6,1,1)
Введите индекс начальной вершины (от 1 до 6): 1
Введите индекс конечной вершины (от 1 до 6): 4
Запрос отправлен на сервер

=== Результат от сервера ===
Успех! Кратчайший путь: 3
Маршрут: 1 -> 2 -> 3 -> 4

Введите команду: PASS
```

Рис. 3: Результат тестирования

На рис. 4 приведен результат теста соединения по протоколу TCP с загрузкой графа из файла.

```
Тест: TCP: ввод из файла, N=53
spawn ./server 8080
spawn ./client 127.0.0.1 8080 TCP
=== Клиент поиска кратчайшего пути в графе ===
Сервер: 127.0.0.1:8080
Протокол: TCP
Успешно подключен к серверу по TCP

Введите 'help' для просмотра доступных команд

Введите команду: file
Введите имя файла: /tmp/test_graph_53.txt
Запрос отправлен на сервер

=== Результат от сервера ===
Успех! Кратчайший путь: 2
Маршрут: 1 -> 6 -> 5

Введите команду: PASS
```

Рис. 4: Результат тестирования

На рис. 5 приведен результат теста соединения по протоколу UDP с вводом графа с клавиатуры.

```

Тест: UDP: ввод с клавиатуры, N=100
spawn ./server 8080
spawn ./client 127.0.0.1 8080 UDP
=== Клиент поиска кратчайшего пути в графе ===
Сервер: 127.0.0.1:8080
Протокол: UDP
UDP клиент запущен

Введите 'help' для просмотра доступных команд

Введите команду: input

=== Ввод данных о графе ===
Введите количество вершин в графе (целое число от 6 до 100): 100
Перечислите рёбра графа в формате (вершина1, вершина2, вес):
Пример: (1,2,5) (2,3,3) (1,3,10)
Введите рёбра: (1,2,1) (2,3,1) (3,4,1) (4,5,1) (5,6,1) (6,1,1)
Введите индекс начальной вершины (от 1 до 100): 1
Введите индекс конечной вершины (от 1 до 100): 2
Отправка данных на сервер...
Попытка 1 из 3
Данные отправлены
Подтверждение получено

=== Результат от сервера ===
Успех! Кратчайший путь: 1

```

Рис. 5: Результат тестирования

На рис. 6 приведены результаты теста с обработкой графов, не соответствующих граничным условиям.

<pre> Тест: TCP: N=5 (недопустимо) spawn ./server 8080 spawn ./client 127.0.0.1 8080 TCP === Клиент поиска кратчайшего пути в графе === Сервер: 127.0.0.1:8080 Протокол: TCP Успешно подключен к серверу по TCP Введите 'help' для просмотра доступных команд Введите команду: input === Ввод данных о графе === Введите количество вершин в графе (целое число от 6 до 100): 5 Ошибка: введите число от 6 до 100 Введите количество вершин в графе (целое число от 6 до 100): </pre>	<pre> Тест: TCP: N=101 (недопустимо) spawn ./server 8080 spawn ./client 127.0.0.1 8080 TCP === Клиент поиска кратчайшего пути в графе === Сервер: 127.0.0.1:8080 Протокол: TCP Успешно подключен к серверу по TCP Введите 'help' для просмотра доступных команд Введите команду: input === Ввод данных о графе === Введите количество вершин в графе (целое число от 6 до 100): 101 Ошибка: введите число от 6 до 100 Введите количество вершин в графе (целое число от 6 до 100): </pre>
--	--

Рис. 6: Результаты тестирования

На рис. 7 приведен результат тестирования UDP соединения в случае, когда сервер недоступен. Тест демонстрирует, что клиент пытается отправить данные на сервер 3 раза с таймаутом 3 секунды. После исчерпания попыток клиент завершает свою работу.

На рис. 8 приведен результат теста, проверяющего корректность расчета кратчайшего пути.

```

Тест: UDP: сервер недоступен
spawn ./client 127.0.0.1 8080 UDP
=== Клиент поиска кратчайшего пути в графе ===
Сервер: 127.0.0.1:8080
Протокол: UDP
UDP клиент запущен

Введите 'help' для просмотра доступных команд

Введите команду: input

=== Ввод данных о графе ===
Введите количество вершин в графе (целое число от 6 до 100): 6
Перечислите рёбра графа в формате (вершина1, вершина2, вес):
Пример: (1,2,5) (2,3,3) (1,3,10)
Введите рёбра: (1,2,1) (2,3,1) (3,4,1) (4,5,1) (5,6,1) (6,1,1)
Введите индекс начальной вершины (от 1 до 6): 1
Введите индекс конечной вершины (от 1 до 6): 2
Отправка данных на сервер...
Попытка 1 из 3
Данные отправлены
Подтверждение не получено в течение 3 секунд
Повторная отправка...
Попытка 2 из 3
Данные отправлены
Подтверждение не получено в течение 3 секунд
Повторная отправка...
Попытка 3 из 3
Данные отправлены
Подтверждение не получено в течение 3 секунд
Потеряна связь с сервером
Завершение работы клиента...
PASS

```

Рис. 7: Результат тестирования

```

Тест: TCP: проверка правильности кратчайшего пути (ожидаем путь длины 20: 1->3->6->5)
spawn ./server 8080
spawn ./client 127.0.0.1 8080 TCP
=== Клиент поиска кратчайшего пути в графе ===
Сервер: 127.0.0.1:8080
Протокол: TCP
Успешно подключен к серверу по TCP

Введите 'help' для просмотра доступных команд

Введите команду: input

=== Ввод данных о графе ===
Введите количество вершин в графе (целое число от 6 до 100): 6
Перечислите рёбра графа в формате (вершина1, вершина2, вес):
Пример: (1,2,5) (2,3,3) (1,3,10)
Введите рёбра: (1,2,7) (1,3,9) (2,3,10) (2,4,15) (3,4,11) (3,6,2) (4,5,6) (5,6,9)
Введите индекс начальной вершины (от 1 до 6): 1
Введите индекс конечной вершины (от 1 до 6): 5
Запрос отправлен на сервер

=== Результат от сервера ===
Успех! Кратчайший путь: 20
Маршрут: 1 -> 3 -> 6 -> 5

Введите команду: PASS

Все тесты пройдены успешно!

```

Рис. 8: Результат тестирования